

Learning & Blocker Journal

The Meridian Pivot — Independent Learning & Autonomy Log

Unfamiliar Tool: Cloudflare Workers (serverless functions), including Cron Triggers and Workers KV

Constraint: No teammate or instructor how-to help during the solo recon window; genuinely new technology, not something previously half-known

Resources Used: Cloudflare's official Workers documentation and quickstart pages; error messages searched directly as they occurred

1. Why This Tool

Serverless functions were chosen for the solo recon phase specifically because they offer a fast, visible feedback loop — code can be deployed and tested against a live URL within minutes — while still connecting naturally to the other unfamiliar-tool categories (message queues, webhooks, retry/backoff) once caching and event-driven routes were introduced later in the sprint. This made it possible to build real troubleshooting depth rather than a shallow “hello world” exercise.

2. Time-to-Completion

Phase	Time-boxed	Actual
Solo recon — Cloudflare Workers basics	2 days (per sprint)	Learned incrementally, step by step, over multiple sessions
Steps 1–2 (request parsing, JSON response)	Part of Day 1–2 window	Multiple iterations — syntax errors caught and fixed before working
Step 3 (external API call)	Part of Day 1–2 window	Required swapping the assumed API for a real public one after discovering the original endpoint didn't exist
Step 4 (error handling)	Part of Day 1–2 window	Verified via two deliberate failure injections (404 and unreachable-domain cases)
Day 3 (polling, caching, query endpoint)	1 day	Cron trigger, KV namespace, and /stock route all built and independently verified
Day 4 (pivot — async webhook model)	48 hrs, no extension	Full pending/checked_in state machine built and tested across 4 attendees

3. Blocker Log

The entries below are recorded in the order they were actually encountered, including the ones that came from genuine misunderstandings rather than typos — both are left in, since the sprint's own grading criteria treats troubleshooting autonomy as evidence, not just the final working state.

Blocker 1: Unquoted string literal in Response()

Symptom: Deploy failed silently / code would not run when returning a plain message from the fetch handler.

Initial Attempt: The very first version of this route built a new Request object pointing at a hardcoded local dev address and never returned anything at all. The attempt right after that added a return statement — ``new Response(Your order is under review)`` — but wrote the message without quotes.

Root Cause: With no return statement, the handler produced no response at all. Once a return was added, the unquoted words were parsed by JavaScript as variable name references rather than plain text, since only quoted text is treated as a string literal — so it failed before ever reaching a working response.

Fix: Wrapped the text in double quotes so it was treated as a string literal.

Lesson: A missing pair of quotes is invisible at a glance but fatal to the parser — worth reading return statements character by character when something “should” work but doesn't.

Blocker 2: Reading the wrong URL for query parameters

Symptom: ``searchParams.get()`` always returned null, even when a query parameter was clearly present in the address bar.

Initial Attempt: Parsed a fixed, hardcoded address — ``new URL("https://northstar-lovat-two.vercel.app/")`` — instead of the real incoming request, and looked up a parameter literally named "Wrong size/Don't like it" rather than a real parameter key.

Root Cause: Because the URL being parsed was hardcoded, it never reflected whatever address a visitor actually used to reach the Worker — appending real query parameters to the live URL had no effect on it. The parameter name being searched for also wasn't something any real visitor would type, so ``get()`` was guaranteed to return null regardless.

Fix: Switched to parsing ``request.url`` so the code reads the real address a visitor used to reach the Worker.

Lesson: Confirmed that ``request.url`` is the live, per-visit information, whereas a hardcoded string will always evaluate to the same thing no matter what a user actually does.

Blocker 3: Duplicate `export default` / duplicate `scheduled` property

Symptom: Later versions of the file either failed to deploy or silently ran only one of two intended behaviors.

Initial Attempt: Wrote two separate ``async scheduled(...)`` blocks inside the same exported object — the original one with just the `console.log`, and a new one added to introduce the KV ``put()`` call — expecting both to execute.

Root Cause: JavaScript objects can only have one property with a given name; writing two separate ``scheduled(...)`` blocks (or two ``export default`` blocks) meant the second silently overwrote the first rather than raising an error or combining the two.

Fix: Merged the intended logic into a single ``scheduled`` (and later, a single ``fetch``) function instead of duplicating the property name.

Lesson: Some bugs don't throw errors — they just quietly discard code. Worth checking object structure, not just syntax, when behavior looks partially missing.

Blocker 4: Field names copied from the wrong API's shape

Symptom: An external API call ran without error but returned an object full of ``undefined`` fields.

Initial Attempt: Pointed ``apiUrl`` at a personal Vercel-hosted frontend project's ``/api/data`` path, and mapped the response using GitHub API field names (``full_name``, ``stargazers_count``) copied over from an unrelated reference example.

Root Cause: The frontend project being called had no backend at all — it was a pure client-side app that kept everything in local component state, so ``/api/data`` was never a real endpoint and could not return anything

meaningful. Separately, even once a real API was used, the field names assumed (from GitHub's API shape) didn't match that API's actual response shape, so every mapped field came back `undefined`.

Fix: Visited the real endpoint directly in a browser first to inspect its actual JSON, then rewrote the field mapping to match the real response.

Lesson: A function can run successfully and still be wrong. Checking the real shape of a response before writing code against it prevents a whole class of silent, undetected bugs.

Blocker 5: `jsonResponse` called but never defined

Symptom: Deploy succeeded, but the route crashed at runtime whenever the error-handling branch was hit.

Initial Attempt: Kept a call to `jsonResponse(...)` , a custom helper function that had existed in an earlier, separate draft of the file (one with its own routing and a `jsonResponse` helper defined at the bottom), while the rest of the merged file had already moved to using the built-in `Response.json(...)` directly.

Root Cause: The helper function's actual definition was never carried into the merged file — only the call to it was. Calling a function that doesn't exist anywhere in the file throws a runtime “not defined” error the moment that code path executes.

Fix: Replaced the remaining `jsonResponse(...)` calls with `Response.json(...)` , matching the pattern used everywhere else in the file.

Lesson: Partial refactors are a real source of bugs — changing a pattern in most places but not all of them can leave dead references that only surface when that specific code path actually runs.

Blocker 6: Cron trigger appeared to never fire

Symptom: Watched `wrangler tail` for over ten minutes with no log output, despite a successful deploy confirming the trigger was registered.

Initial Attempt: Assumed that waiting roughly 5 minutes from whenever the terminal started watching would be enough to see the schedule fire, and concluded after about 12 minutes of silence that the trigger must be broken.

Root Cause: Cron schedules fire on fixed clock marks (e.g., :00, :05, :10), not a fixed number of minutes after the terminal starts watching — the observation window simply hadn't crossed the next real firing time yet, so the “wait 5 minutes” assumption didn't match how the schedule actually worked.

Fix: Checked the Cloudflare dashboard's Triggers tab to see the actual next scheduled fire time, then waited specifically until that clock time.

Lesson: “Nothing happened yet” and “this is broken” are different conclusions. Checking the system's own stated schedule before assuming a defect avoided a wrong diagnosis.

Blocker 7: KV binding name mismatch (Error 1101)

Symptom: A new route threw a worker exception (Cloudflare Error 1101) on every request.

Initial Attempt: Referenced `env.STOCK\_DATA` in the new attendee check-in logic, assuming it referred to an already-bound namespace, without ever creating or declaring a namespace by that name in `wrangler.jsonc` .

Root Cause: No KV namespace called `STOCK\_DATA` existed anywhere in the config, so `env.STOCK\_DATA` was simply `undefined` at runtime; calling `.get()`/`.put()` on `undefined` threw the exception behind the generic Error 1101 page.

Fix: Created a dedicated `ATTENDEE\_CACHE` KV namespace, bound it explicitly in `wrangler.jsonc` , and updated every code reference to match the real binding name.

Lesson: A binding existing in one file (code) means nothing if it isn't also declared and named identically in the other (config). The two have to agree exactly.

Blocker 8: New route nested inside another route's braces

Symptom: A newly added `/status`` route never responded — visiting it fell through to the parent route's logic instead.

Initial Attempt: Pasted the entire new `if (url.pathname === "/status")`` block directly above the "mark as pending" logic, but still inside the closing brace of the `/checkin`` block, rather than after it.

Root Cause: Because it was nested inside `/checkin``'s block, the `/status`` condition could only be reached while `url.pathname`` was already `"/checkin"` — a state under which it could never also equal `"/status"` — making the block permanently unreachable no matter what URL was visited.

Fix: Moved the `/status`` block out to sit as a sibling of `/checkin``, both closing fully before the next route begins.

Lesson: Brace placement is logic, not just formatting — code can be free of typos and still be structurally unreachable.

Blocker 9: Webhook logic rejected its own success case

Symptom: The `/webhook`` route always returned an error, even for an attendee who should have been confirmable.

Initial Attempt: After the `checked_in`` rejection check, added an extra guard — `if (existing.status === "pending") { return error }`` — intending it as another safety check before allowing the update.

Root Cause: By that point in the function, `checked_in`` had already been ruled out by the check just above, so `pending`` was the only status value that could still be true — meaning this new check was always true, always returned the error, and the real `.put()`` update sitting beneath it could never be reached.

Fix: Removed the incorrect branch, leaving `pending`` as the implicit “safe to confirm” case reached only after ruling out `checked_in`` and “not found.”

Lesson: Adding a check that seems protective can accidentally block the one case a function exists to handle. Worth tracing which values can actually still be true at each point before adding a new condition.

Blocker 10: Stale read after a confirmed status change

Symptom: A `/checkin`` request for an already-confirmed attendee briefly returned their old `pending`` status instead of `checked_in``.

Initial Attempt: Initially assumed this meant the earlier `checked_in`` update had somehow failed, been overwritten, or reverted, since nothing else could obviously explain an attendee's status moving backward.

Root Cause: No code path in the Worker ever writes `pending`` over an existing `checked_in`` record, so a genuine reversion wasn't actually possible from the code's logic. Cloudflare KV is eventually consistent instead — a write can take a short time to propagate across edge locations, so a read immediately after a write can occasionally return a stale value from a different edge location that hadn't yet received the update.

Fix: Re-queried `/status`` for the same attendee shortly after; the correct `checked_in`` value was returned, confirming the delay was transient rather than a logic defect.

Lesson: Not every unexpected result is a bug in the code — some are inherent properties of the infrastructure being used, and are worth documenting as a real trade-off rather than chased as a phantom defect.

5. Summary

Across the sprint, the recurring pattern was the same: most bugs were not failures of understanding a new concept, but small structural mistakes — a missing quote, a misplaced brace, a mismatched name across two files, an outdated field assumption — that only became visible by actually deploying and testing against a live endpoint rather than reading the code and assuming it was correct. That discipline (deploy, test, verify with real output, only then move on) was the single most repeated and most useful habit built during this recon phase.